CSL7360: Computer Vision

Assignment 2

Cost-Efficient Segmentation under a Compute Budget

Objective

Create a pipeline that labels every pixel of a photo as either main object or background, using on average at most 50 ms of CPU time on a single core per photo. You can follow **Track A**, which uses classical methods (clustering followed by a graph cut), or **Track B**, which uses a small CNN. Your goal is to make the Cost-Efficient Segmentation (CES) score, defined below, as high as possible. **No boilerplate code is provided for this assignment. You are responsible for structuring the pipeline yourself.**

1. Data Collection

a. **DUTS Dataset:** Download DUTS from `http://saliencydetection.net/duts/`. Its masks mark the main object or objects of each photo in white. It has 10,553 training photos (DUTS-TR) and 5,019 test photos (DUTS-TE). Split DUTS-TR into your own training and validation sets and make all design choices on the validation set. Never use DUTS-TE for training, tuning or deciding when to stop training.

b. **Resizing:** Resize every photo to 256×256 with `cv2.INTER_AREA` and every mask with `cv2.INTER_NEAREST`. A mask pixel is object if its value is at least 128. All metrics are computed on these 256×256 masks. Squashing a photo that is not square is intended.

c. **Your Own Photos:** Take 5 to 10 phone photos around your college or hostel, each showing one clear everyday object such as a water bottle, a bag, a cycle, a chair, a pair of shoes, a plant or a campus cat. Together they should cover at least three of these five conditions: both indoor and outdoor scenes, both daylight and tube light, both near and far objects, a cluttered background, and partial occlusion. Resize each photo with the rule above, then draw its mask on the resized photo with any tool you like, such as Paint, GIMP, LabelMe or your own script. Save masks as PNGs with the object in white (255) and the background in black (0). If your tool leaves grey pixels along the edges, the 128 threshold above decides them. Use these photos only for testing.

2. Rules, Budget and Score

a. **Libraries:** You may use the Python standard library, and PyTorch, NumPy, OpenCV and Matplotlib only for file I/O, data loading (`torch.utils.data`), resizing, colour conversion, filtering with `cv2.filter2D`, plotting, tensors, autograd, optimizers, and layers or functions from `torch.nn` and `torch.nn.functional` (including `F.max_pool2d`). You are **NOT allowed** to use pretrained weights or library code for segmentation, superpixels, clustering, max flow, morphology or metrics (such as `torchvision.models`, `sklearn`, `skimage`, `scipy.ndimage`, `networkx`, `PyMaxflow`, `cv2.kmeans`, `cv2.grabCut`, `cv2.dilate` or `cv2.erode`), or ONNX Runtime and other inference engines. TorchScript and `torch.compile` are allowed. If you are unsure about a function, please ask.

b. **Timing:** The timed part of your pipeline starts with a 256×256 photo in memory and ends with its mask, a 256×256 array of 0s and 1s, in memory. Every step in between counts, including any downscaling and colour conversion. Reading and writing files, resizing the original photo, loading parameters and training are not timed. You may train on a GPU, but the timed pipeline must run on the CPU with a single thread and no extra processes (no multiprocessing or subprocesses). Before importing NumPy, OpenCV or PyTorch, set `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `VECLIB_MAXIMUM_THREADS` to `"1"`, then call `cv2.setNumThreads(1)` and `torch.set_num_threads(1)`. Run your pipeline once on any photo before you start timing, since the first run is slow while libraries load. Then, on

your own laptop, segment the 5,019 DUTS-TE photos one at a time and add up the time each takes with `time.process_time()`. T_{CPU} is this total divided by 5,019. It must be **at most 50 ms**, otherwise your CES counts as zero. The limit is generous, and TAs may rerun `predict.py` on some submissions to check reported times.

c. **Model Size:** S_{disk} is the size in bytes of your saved parameter file, from `os.path.getsize()`. All learned values must be in this file, not written into your code. You may store weights at lower precision, such as float16. The default `torch.save` and `np.savez` formats are fine, but you may not compress the file further (for example with gzip or `np.savez_compressed`).

d. **Score:** The quality Q combines region overlap (mIoU) with edge accuracy (Boundary F1), and Cost-Efficient Segmentation (CES) score takes off a little for time and file size:

$$Q = 0.70\,\text{mIoU} + 0.30\,\text{BF1}$$

$$\text{CES} = \begin{cases} Q\left[1 - 0.20\,\dfrac{T_{\text{CPU}}}{50\text{ ms}} - 0.10\min\left(1, \dfrac{S_{\text{disk}}}{4\text{ MiB}}\right)\right] & \text{if } T_{\text{CPU}} \le 50\text{ ms} \\ 0 & \text{otherwise} \end{cases}$$

Here 4 MiB is 4,194,304 bytes. For example, $Q = 0.70$ at 20 ms with a 0.5 MiB file gives $0.70 \times (1 - 0.08 - 0.0125) = 0.635$. CES is computed on all of DUTS-TE, the one set that is the same for everyone. Also report your validation and captured photo metrics, which show how well your method generalises.

3. Implementation Requirements

Track A: Classical Computer Vision (from scratch)

You may use NumPy or PyTorch, for example autograd to train your region model, but write the clustering and the max flow yourself.

- **Clustering into Regions:** Describe each pixel by its colour and its (x, y) position, optionally with texture filter responses, so that clusters form compact regions. Group the pixels with K-Means, a GMM trained with EM, or Mean-Shift, choosing the fastest one that suits your pipeline.
- **Region Probabilities:** Use your training set to learn a probability P_p that region p is object, and clip it to $[0.001, 0.999]$. For example, you could record how often each image position is object, build colour histograms of object and background pixels, or measure how much a region touches the image border, and then combine such clues with a logistic regression you train yourself.
- **Graph Cut:** Build a graph with one node per region, a source node for object and a sink node for background. Connect the source to region p with capacity $-\log(1 - P_p)$ and region p to the sink with capacity $-\log P_p$, both times the region's pixel count. Cutting the sink edge keeps p on the source side and pays the object cost $-\log P_p$, and cutting the source edge pays the background cost. Two regions touch if any of their pixels are directly above, below, left or right of each other. Join touching regions in both directions with an edge of weight λW_{pq} times their shared border length (the number of touching pixel pairs), where $W_{pq} = \exp(-\|f(p) - f(q)\|^2 / 2\sigma_d^2)$ and f is the mean colour of a region, so that similar neighbours prefer the same label. Tune λ and σ_d on your validation set, starting from $\lambda = 1$ and, for each image, σ_d^2 equal to the average of $\|f(p) - f(q)\|^2$ over its touching regions, plus a small constant such as 10^{-6} so that it is never zero. Find the minimum cut with your own max flow algorithm, such as Edmonds-Karp. Regions on the source side of the cut are object.
- **Efficiency:** Clustering and max flow in Python over all 65,536 pixels will not fit in 50 ms. Instead, cluster a smaller copy of the photo (for example 64×64) into about 100 to 200 regions, run the graph cut on those regions, and enlarge the final mask to 256×256.

Track B: Fully Convolutional Network (from scratch in PyTorch)

- **Network:** Build a small fully convolutional network (FCN) as covered in class, from convolutions (including any of 1×1 and depthwise separable ones), pooling, batch norm, ReLU, and transpose convolutions or bilinear upsampling. The encoder shrinks the image, the decoder brings it back to full size, and skip connections pass encoder features to the decoder, either by adding them as in FCN-8s or by concatenating them as in U-Net. Train it from scratch with pixel wise cross entropy, optionally adding a Soft Dice loss.
- **Efficiency:** Choose the size of the network and the resolution it works at so that the pipeline stays under 50 ms.

Evaluation Metrics: Both Tracks (from scratch)

- **mIoU:** Add up one 2×2 confusion matrix over all images in a set, compute the IoU (intersection over union) for background and for object, and average the two.
- **Boundary F1:** BF1 checks whether the outline of your mask is in the right place.
  1. A boundary pixel is an object pixel with a background pixel directly above, below, left or right of it. Pixels outside the image count as background.
  2. A predicted boundary pixel is *correct* if a true boundary pixel lies within its 5×5 window, which allows for small annotation errors. A true boundary pixel is *found* if a predicted boundary pixel lies within its 5×5 window.
  3. Precision P is the fraction of predicted boundary pixels that are correct, and recall R is the fraction of true boundary pixels that are found. $\text{BF1} = 2PR/(P + R)$, or 0 if $P + R = 0$.
  4. Compute BF1 per image and average over the set. If neither mask has boundary pixels, BF1 is 1, and if only one does, it is 0. Looping over every pixel and its window in Python is too slow for 5,019 images, so find a faster way to do this check.

Deliverables

- **Captured Photos:** A folder named `captured` with three subfolders: `original` for the photos as taken, `images_256` for the resized photos as PNGs, and `masks_256` for your masks as PNGs. Use the same base name in all three, for example `original/item1.jpg`, `images_256/item1.png` and `masks_256/item1.png`.
- **Codebase and Parameters:** Clean, well documented code for training, evaluation and timing, together with your saved parameter file.
- `predict.py`: A script that runs as `python predict.py --image photo.jpg --out mask.png`. It loads your parameters, resizes the photo with the rule from Section 1b, segments it and saves the mask as a 0 and 255 PNG. It then times 20 more runs on the same photo, since the first run is slow while libraries load, and prints exactly two lines with the average CPU time per run in milliseconds and the file size in bytes:
  ```
  T_CPU_ms:  12.34
  S_disk_bytes:  425767
  ```
- **Report:** Describe your method, how you kept it fast, your training and validation split sizes and your CPU model. Give one table with mIoU, Boundary F1 and Q on your validation set, DUTS-TE and your own photos, together with T_{CPU}, S_{disk} and CES. For every captured photo, show the true and predicted masks side by side as semi transparent overlays. Finally, discuss at least two failure cases and their likely causes, such as poor lighting, occlusion, clutter or a small object.